



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机组成原理 COMPUTER ORGANIZATION

本课件内容参考自David A. Patterson与John L. Hennessy的教材《Computer Organization and Design: The Hardware/Software Interface》官方配套课件。



Computer Organization

Lecture 13

Virtual Memory



Outline

- Virtual Memory
- Memory hierarchy summary
- Virtual Machines



Computer Organization

Virtual Memory

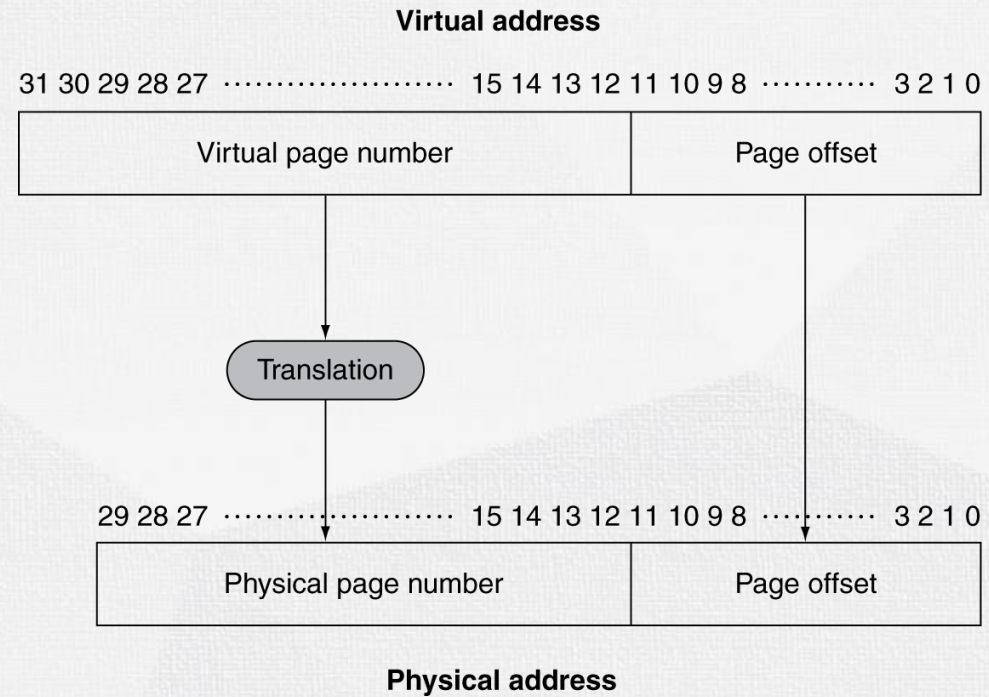
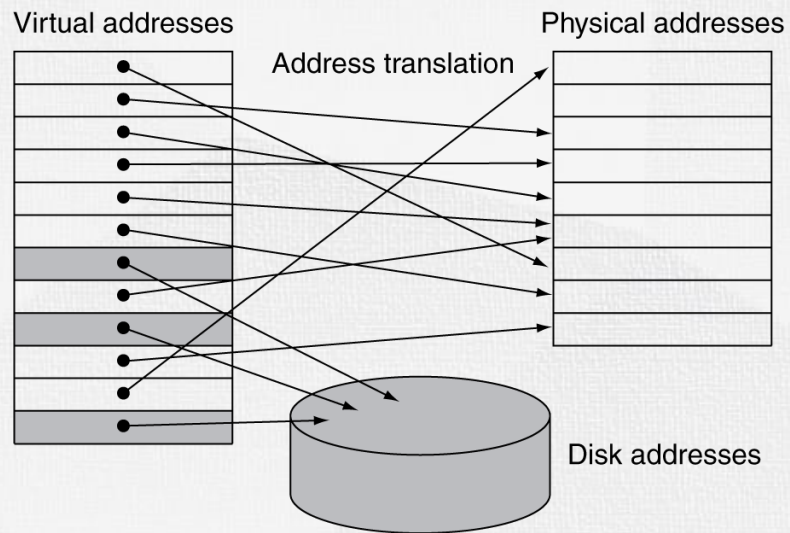


Virtual Memory

- Use main memory as a “cache” for secondary (disk) storage
 - Managed jointly by CPU hardware and the operating system (OS)
- Programs share main memory
 - Each gets a private virtual address space holding its frequently used code and data
 - Protected from other programs
- CPU and OS translate virtual addresses to physical addresses
 - VM “block” is called a **page**
 - VM translation “miss” is called a **page fault**

Address Translation

- Fixed-size pages (e.g., 4K)





Page Fault Penalty

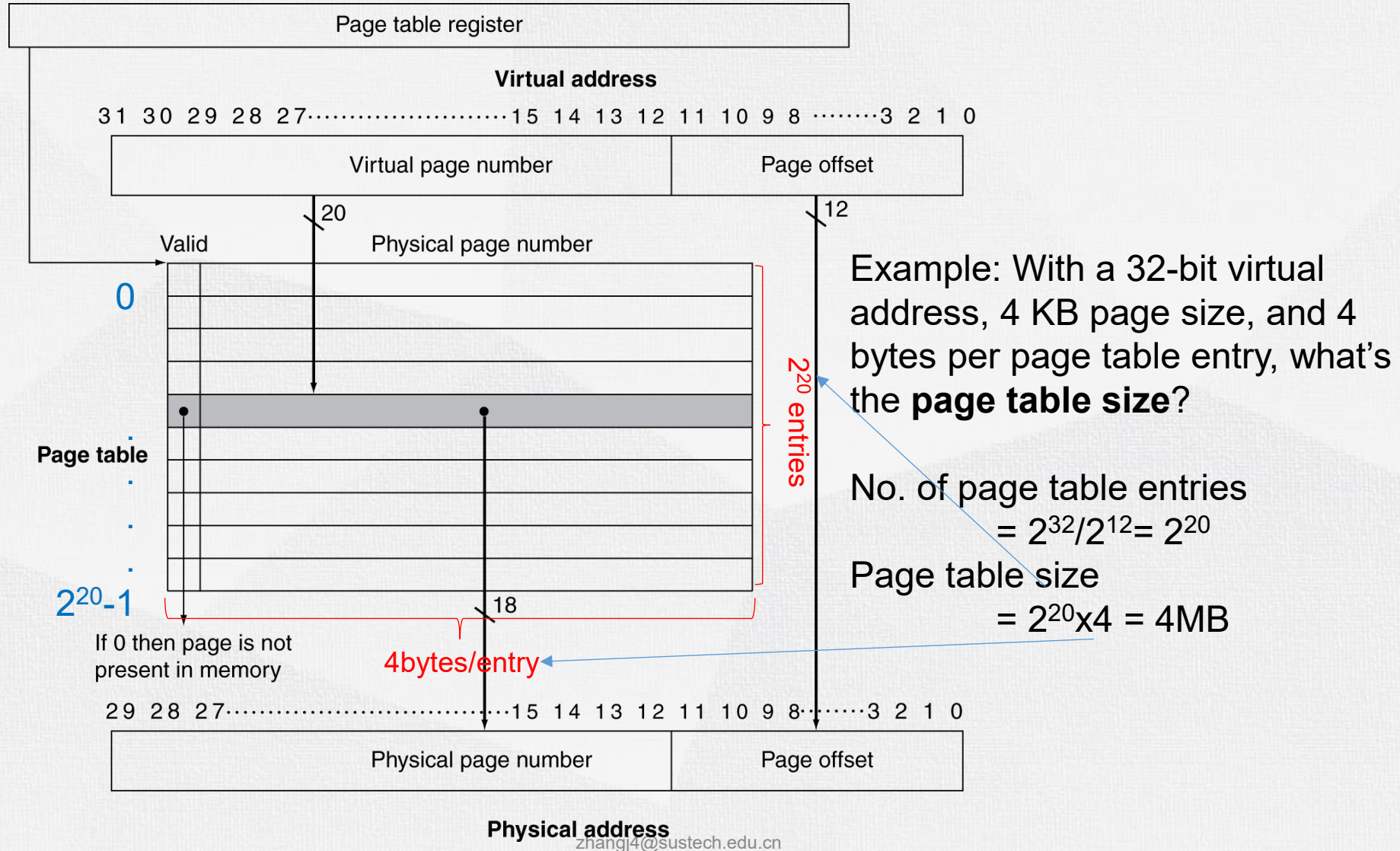
- On page fault, the page must be fetched from disk
 - Takes millions of clock cycles
 - Handled by OS code
- Try to minimize page fault rate
 - Fully associative placement
 - Smart replacement algorithms



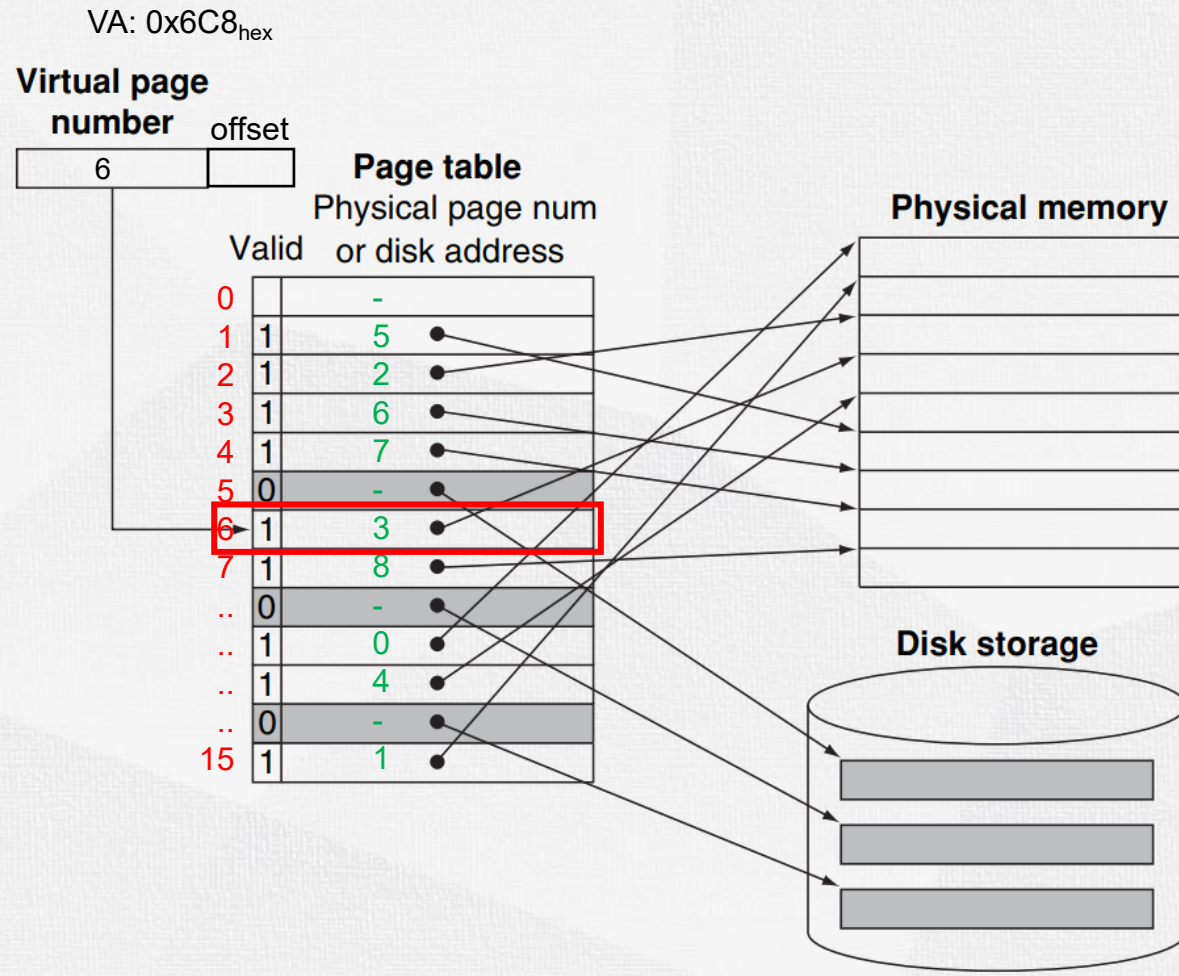
Page Tables

- Stores placement information
 - Array of **page table entries (PTE)**, indexed by **virtual page number**
 - **Page table register** in CPU points to page table in physical memory
- Each program has its page table. **Page table is in memory**
- If page is present in memory
 - PTE stores the **physical page number**
 - Plus other status bits (**referenced bit, dirty bit, ...**)
- If page is not present
 - PTE can refer to location in swap space on disk

Translation Using a Page Table



Mapping Pages to Storage



Example: **16 virtual pages, 256B/page**, virtual address: 12 bits.

To what physical address does virtual address 0x6C8 map to?

page offset: $\log_2(256B) = 8$ bits

0x6C8_{hex} = 011011001000
vpn offset

Virtual page number = 6
 Physical page number = 3
 Physical address =

1111001000 = 0x3C8_{hex}
ppn offset



Replacement and Writes

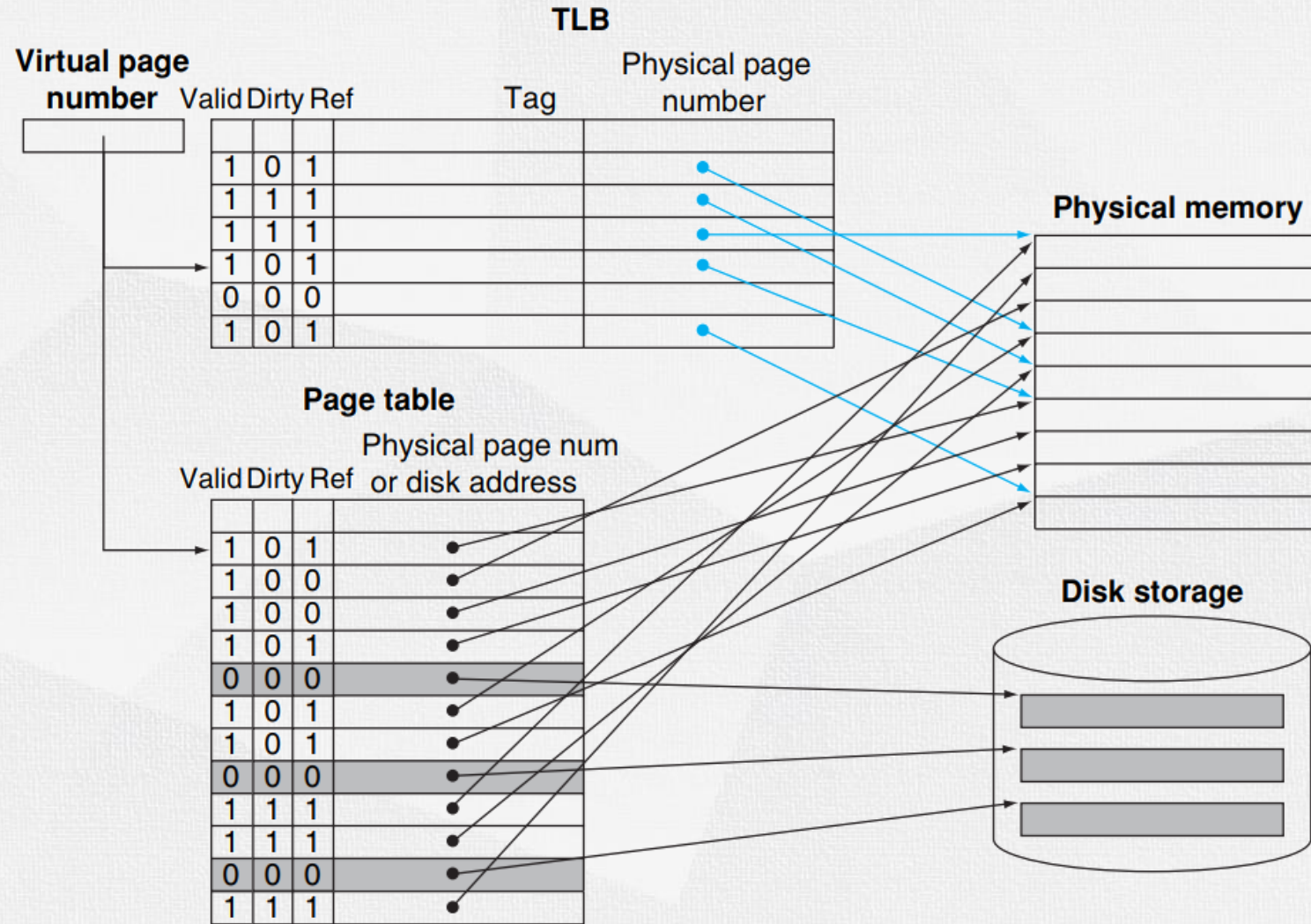
- To reduce page fault rate, prefer least-recently used (LRU) replacement
 - **Reference bit** (aka **use bit**) in PTE set to 1 on access to page
 - Periodically cleared to 0 by OS
 - A page with reference bit = 0 has not been used recently
- Disk writes take millions of cycles
 - Block at once, not individual locations
 - Write through is impractical
 - Use write-back
 - **Dirty bit** in PTE set when page is written



Fast Translation Using a TLB

- Problems of Page Table
 - Access to page table is too slow
 - First access the PTE (one main memory access)
 - Then the actual memory access for data
 - Page table is too big
- But access to page tables has good locality
 - So use a fast cache of PTEs within the CPU
 - Called a **Translation Look-aside Buffer (TLB)**
 - Typical: 16–512 PTEs, 0.5–1 cycle for hit, 10–100 cycles for miss, 0.01%–1% miss rate
 - Misses could be handled by hardware or software

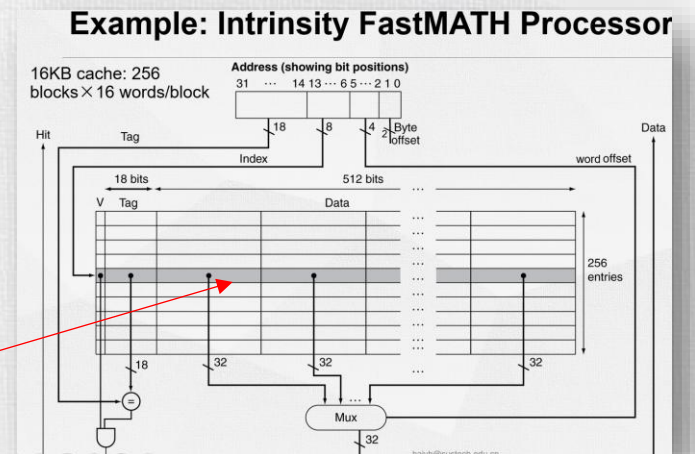
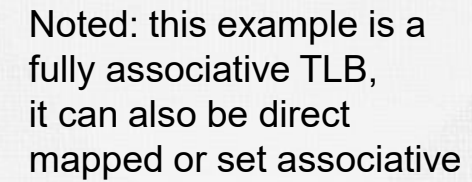
Fast Translation Using a TLB





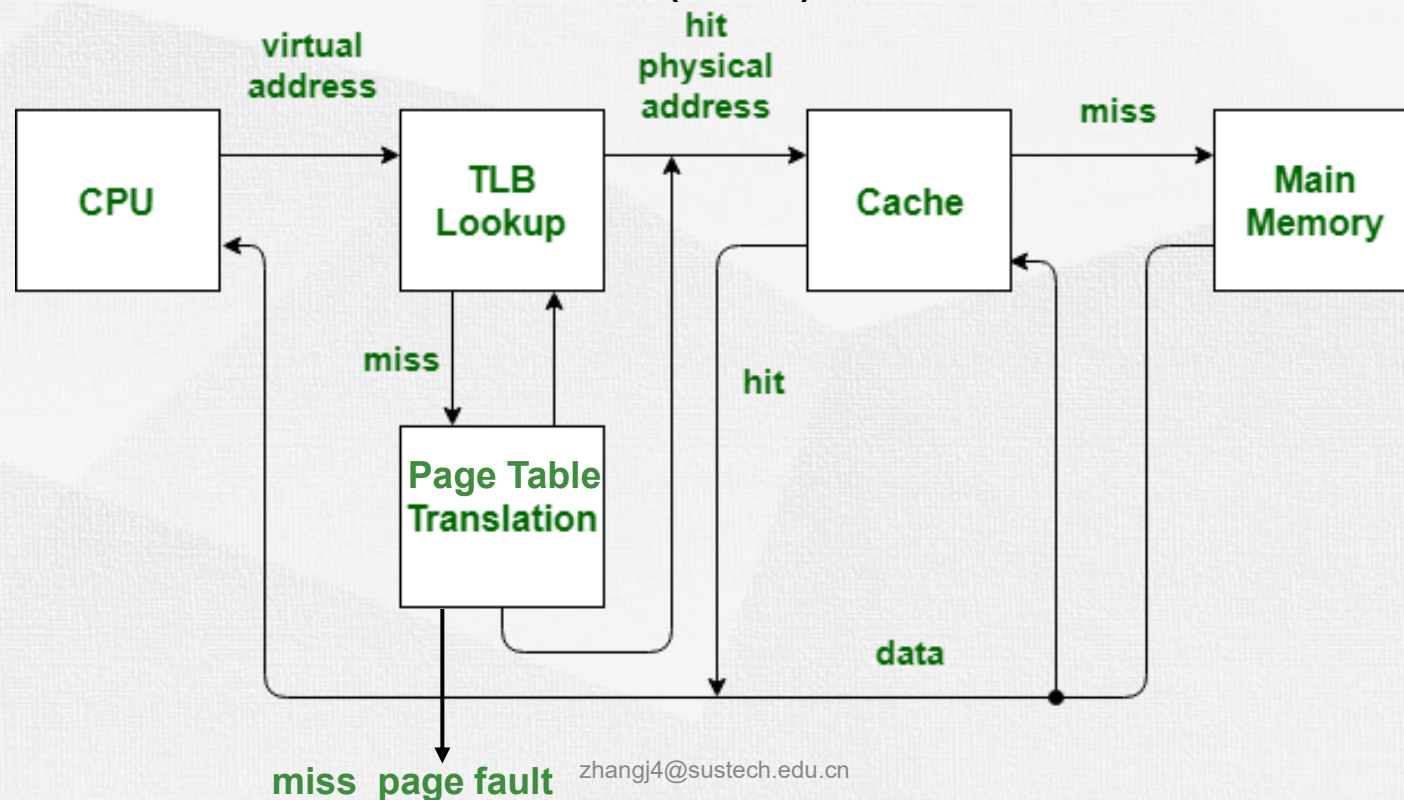
TLB Misses

- If page is in memory
 - Load the PTE from memory and retry
 - Could be handled in hardware
 - Can get complex for more complicated page table structures
 - Or in software
 - Raise a special exception, with optimized handler
- If page is not in memory (page fault)
 - OS handles fetching the page and updating the page table (software)
 - Then restart the faulting instruction



Making Address Translation Practical

- In VM, memory acts like a cache for disk
 - Page table maps virtual page numbers to physical frames
 - Use a page table cache for recent translation
 - => Translation Lookaside Buffer (TLB)





Check Yourself

- Match the definitions between left and right

L1 cache	_____	A cache for a cache
L2 cache	_____	A cache for disks
Main memory	_____	A cache for a main memory
TLB	_____	A cache for page table entries

TLB, Page Table and Cache

- The possible combinations of events in the TLB, virtual memory system, and physically indexed (tagged) cache.

TLB	Page table	Cache	Possible? Condition?
Hit	Hit	Miss	
Miss	Hit	Hit	
Miss	Hit	Miss	
Miss	Miss	Miss	
Hit	Miss	Miss	
Hit	Miss	Hit	
Miss	Miss	Hit	

TLB, Page Table and Cache

- The possible combinations of events in the TLB, virtual memory system, and physically indexed (tagged) cache.

TLB	Page table	Cache	Possible? Condition?
Hit	Hit	Miss	Possible, but page table never checked if TLB hits
Miss	Hit	Hit	Possible, TLB miss but entry found in page table; after retry, data in cache
Miss	Hit	Miss	Possible, TLB miss but entry found in page table; after retry, data miss in cache
Miss	Miss	Miss	Possible, TLB miss and is followed by a page fault
Hit	Miss	Miss	Impossible, not in TLB if page not in memory
Hit	Miss	Hit	Impossible, not in TLB if page not in memory
Miss	Miss	Hit	Impossible, not in cache if page not in memory



Virtual Memory Summary

Memory Protection

- Different tasks can share parts of their virtual address spaces
 - But need to protect against errant access
 - Requires OS assistance
- Hardware support for OS protection
 - Privileged supervisor mode (aka kernel mode)
 - Privileged instructions
 - Page tables and other state information only accessible in supervisor mode
 - System call exception (e.g., ecall in RISC-V)

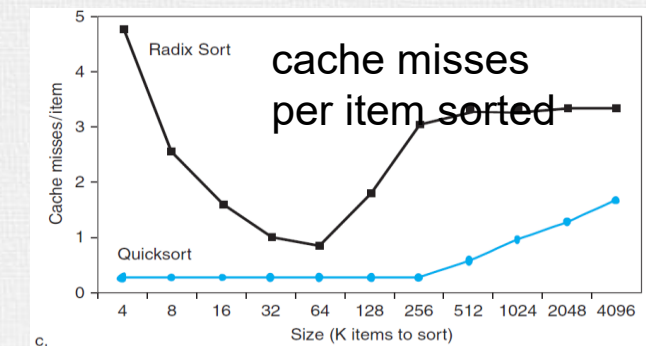
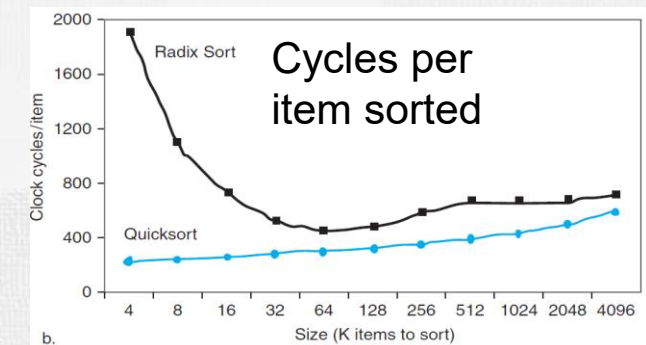
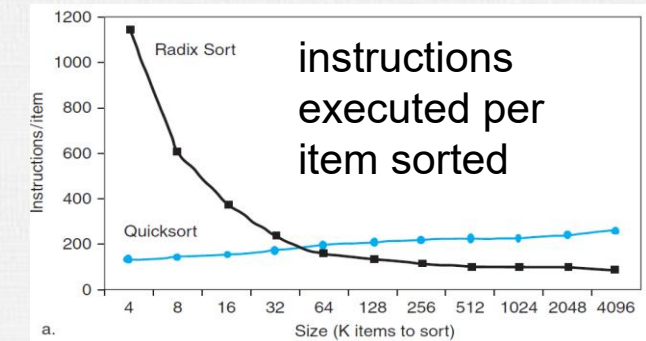


Computer Organization

Memory hierarchy summary

Interactions with Software

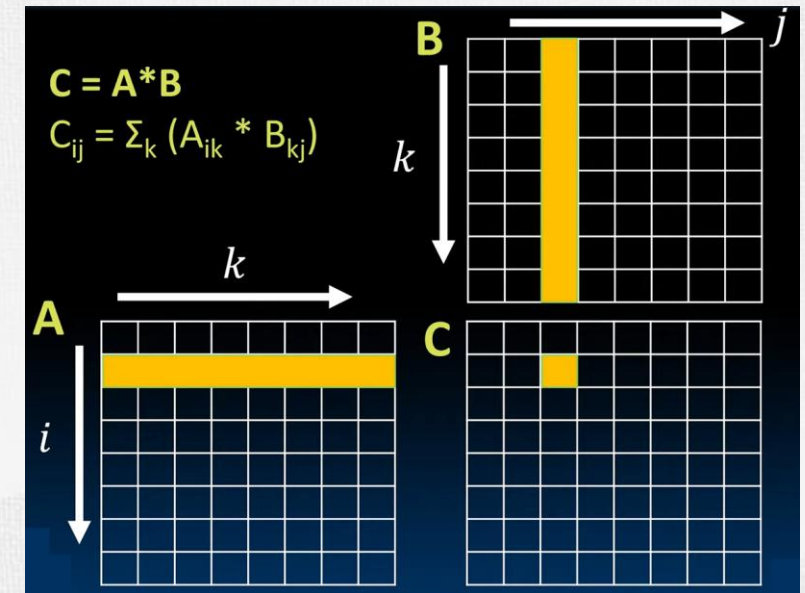
- Misses depend on memory access patterns
 - Algorithm behavior
 - Compiler optimization for memory access
- When #items increase,
 - Radix sort has less instructions
 - But quicksort has less clock cycles
 - Because miss rate of radix sort is higher



Software Optimization via Blocking

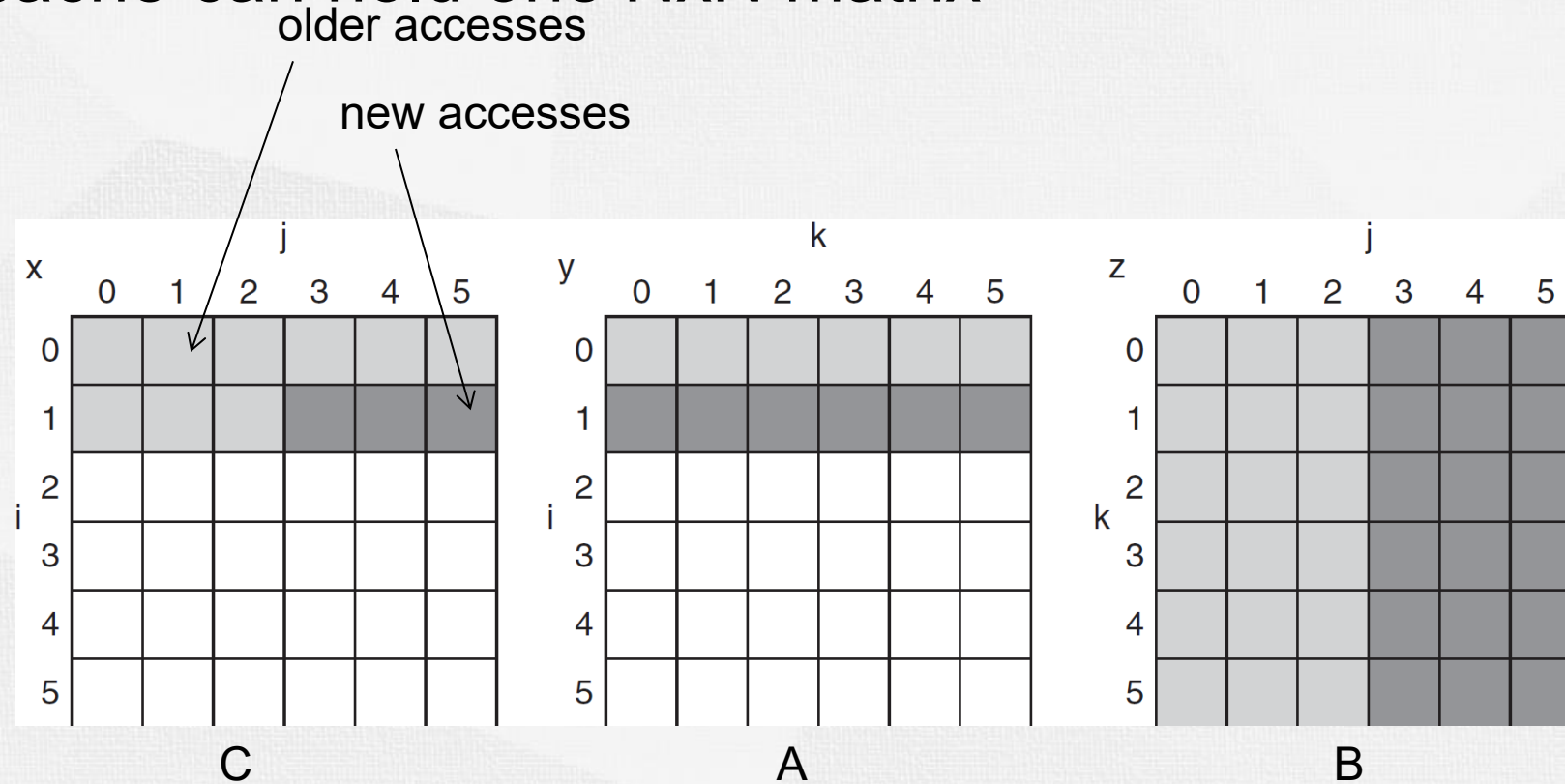
- Goal: maximize accesses to data before it is replaced
- Consider inner loops of DGEMM:
- $N \times N$ matrices

```
for (int j = 0; j < n; ++j)
{
    double cij = C[i+j*n]; // cij = c[i][j]
    for( int k = 0; k < n; k++ )
        cij += A[i+k*n] * B[k+j*n]; // cij += A[i][k]*B[k][j]
    C[i+j*n] = cij; // c[i][j] = cij
}
```

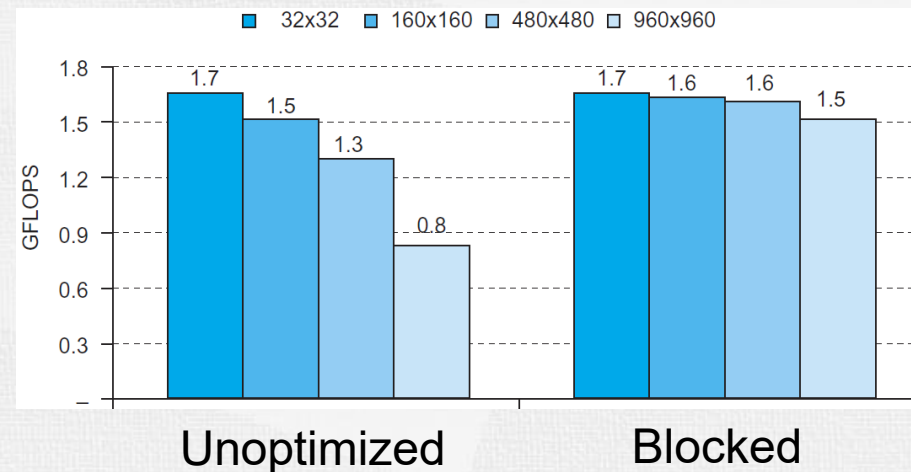
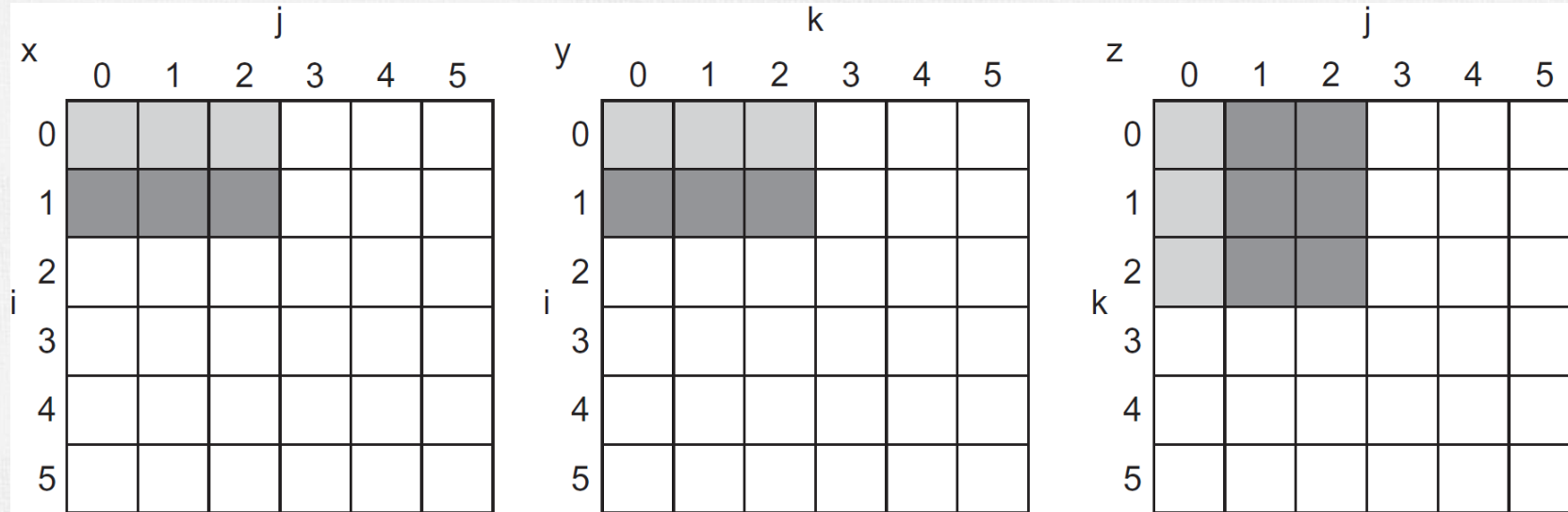


DGEMM Access Pattern

- C, A, and B arrays
- $N = 6$, cache can hold one $N \times N$ matrix



Blocked DGEMM Access Pattern





The Memory Hierarchy Summary

- Common principles apply at all levels of the memory hierarchy
 - Based on notions of caching
- At each level in the hierarchy
 - Block placement
 - Finding a block
 - Replacement on a miss
 - Write policy



Block Placement

- Determined by associativity
 - Direct mapped (1-way associative)
 - One choice for placement
 - n-way set associative
 - n choices within a set
 - Fully associative
 - Any location
- Higher associativity reduces miss rate
 - Increases complexity, cost, and access time

Finding a Block

Associativity	Location method	Tag comparisons
Direct mapped	Index	1
n-way set associative	Set index, then search entries within the set	n
Fully associative	Search all entries	#entries
	Full lookup table	0

- Hardware caches
 - Reduce comparisons to reduce cost
- Virtual memory
 - Full table lookup makes full associativity feasible
 - Benefit in reduced miss rate



Replacement

- Choice of entry to replace on a miss
 - Least recently used (LRU)
 - Complex and costly hardware for high associativity
 - Random
 - Close to LRU, easier to implement
- Virtual memory
 - LRU approximation with hardware support
- Cache
 - Both LRU and Random is ok



Write Policy

- Write-through
 - Update both upper and lower levels
 - Simplifies replacement, but may require write buffer
- Write-back
 - Update upper level only
 - Update lower level when block is replaced
 - Need to keep more state
- Virtual memory
 - Only write-back is feasible, given disk write latency



Sources of Misses

- Compulsory misses (aka cold start misses)
 - First access to a block
- Capacity misses
 - Due to finite cache size
 - A replaced block is later accessed again
- Conflict misses (aka collision misses)
 - In a non-fully associative cache
 - Due to competition for entries in a set
 - Would not occur in a fully associative cache of the same total size

Cache Design Trade-offs

Design change	Effect on miss rate	Negative performance effect
Increase cache size		
Increase associativity		
Increase block size		

Multilevel On-Chip Caches

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	-	Unified (instruction and data)
L3 cache size	-	8 MiB, shared
L3 cache associativity	-	16-way set associative
L3 replacement	-	Approximated LRU
L3 block size	-	64 bytes
L3 write policy	-	Write-back, Write-allocate
L3 hit time	-	35 clock cycles

2-Level TLB Organization

Characteristic	ARM Cortex-A8	Intel Core i7
Virtual address	32 bits	48 bits
Physical address	32 bits	44 bits
Page size	Variable: 4, 16, 64 KiB, 1, 16 MiB	Variable: 4 KiB, 2/4 MiB
TLB organization	1 TLB for instructions and 1 TLB for data Both TLBs are fully associative, with 32 entries, round robin replacement TLB misses handled in hardware	1 TLB for instructions and 1 TLB for data per core Both L1 TLBs are four-way set associative, LRU replacement L1 I-TLB has 128 entries for small pages, 7 per thread for large pages L1 D-TLB has 64 entries for small pages, 32 for large pages The L2 TLB is four-way set associative, LRU replacement The L2 TLB has 512 entries TLB misses handled in hardware



Computer Organization

Virtual Machines



Virtual Machines

- Host computer emulates guest operating system and machine resources
 - Improved isolation of multiple guests
 - Avoids security and reliability problems
 - Aids sharing of resources
- Virtualization has some performance impact
 - Feasible with modern high-performance computers
- Examples
 - IBM VM/370 (1970s technology!)
 - VMWare
 - Microsoft Virtual PC
 - VirtualBox



Virtual Machine Monitor

- Maps virtual resources to physical resources
 - Memory, I/O devices, CPUs
- Guest code runs on native machine in user mode
 - Traps to VMM on privileged instructions and access to protected resources
- Guest OS may be different from host OS
- VMM handles real I/O devices
 - Emulates generic virtual I/O devices for guest



Example: Timer Virtualization

- In native machine, on timer interrupt
 - OS suspends current process, handles interrupt, selects and resumes next process
- With Virtual Machine Monitor
 - VMM suspends current VM, handles interrupt, selects and resumes next VM
- If a VM requires timer interrupts
 - VMM emulates a virtual timer
 - Emulates interrupt for VM when physical timer interrupt occurs



Instruction Set Support

- User and System modes
- Privileged instructions only available in system mode
 - Trap to system if executed in user mode
- All physical resources only accessible using privileged instructions
 - Including page tables, interrupt controls, I/O registers
- Renaissance of virtualization support
 - Current ISAs (e.g., x86) adapting



Concluding Remarks

- Fast memories are small, large memories are slow
 - We really want fast, large memories ☹️
 - Caching gives this illusion 😊
- Principle of locality
 - Programs use a small part of their memory space frequently
- Memory hierarchy
 - L1 cache $\leftrightarrow$ L2 cache $\leftrightarrow$... $\leftrightarrow$ DRAM memory $\leftrightarrow$ disk
- Virtual memory and TLB

Pitfalls

- Byte vs. word addressing
 - Example: 32-byte direct-mapped cache, 4-byte blocks (direct mapped)
 - Byte 36 maps to block 1
 - Word 36 maps to block 4
 - Example: 256-byte cache and 32 bytes per block (direct mapped)
 - Byte address 300 maps to block number 1
0...0 0 0 1 0 0 1 0 1 1 0 0
- Ignoring memory system effects when writing or generating code
 - Example: iterating over rows vs. columns of arrays
 - Large strides result in poor locality

```
for (i = 0; ...) {  
    for (j = 0; ...) {  
        x[i][j] = x[i][j]+y[i][j] } }  
}
```



Interactions with Advanced CPUs

- Out-of-order CPUs can execute instructions during cache miss
 - Pending store stays in load/store unit
 - Dependent instructions wait in reservation stations
 - Independent instructions continue
- Effect of miss depends on program data flow
 - Much harder to analyze
 - Use system simulation